

Quadruped Locomotion with Central Pattern Generators and Deep Reinforcement Learning

Jiří Bláha

Swiss Federal Institute of Technology
CH-1015 Lausanne, Switzerland
SCIPER 415639

Giulia Cortelazzo

Swiss Federal Institute of Technology
CH-1015 Lausanne, Switzerland
SCIPER 344463

Semih Zaman

Swiss Federal Institute of Technology
CH-1015 Lausanne, Switzerland
SCIPER 387929

Abstract—This report investigates quadruped locomotion using a combination of biologically inspired central pattern generators (CPG) and deep reinforcement learning (DRL). In the first part, locomotion controllers based on coupled Hopf oscillators are designed to generate stable and repeatable gait patterns through appropriate phase coupling and parameter tuning. In the second part, the locomotion framework is extended using deep reinforcement learning, where control policies are trained to modulate CPG parameters within a physics-based simulation environment. Key design choices, including observation space, action space, and reward formulation, are discussed and subsequently evaluated.

Index Terms—physics-based simulation, central pattern generators, Hopf network, gait generation, deep reinforcement learning, Markov decision process, observation space, action space, reward function, hyperparameters, locomotion metrics

I. INTRODUCTION

This project focuses on the generation of stable and efficient quadruped locomotion, using either central pattern generators (CPG) or deep reinforcement learning (DRL) as the underlying control paradigm. In the first part, locomotion is generated using biologically inspired CPGs, which produce rhythmic leg motions through coupled nonlinear oscillators and enable intuitive control over gait timing, phase relationships, and foot trajectories. This approach allows for explicit gait design and systematic tuning of locomotion characteristics such as stride duration, duty cycle, and step geometry. In the second part, locomotion controllers are learned using deep reinforcement learning, where control policies are trained to solve velocity-tracking and task-specific locomotion objectives directly from interaction within a physics-based simulation environment. In contrast to the CPG-based approach, DRL does not necessarily rely on explicit gait parametrization, but instead learns control strategies that implicitly encode gait patterns and adaptation to the task at hand. The considered tasks include both flat terrain velocity tracking and locomotion on sloped terrain.

II. METHODOLOGY

This section describes the methodology used to obtain the results presented in Section III, as well as the supplementary results reported in Appendix A. The focus is placed on the key modeling and control design choices underlying both the CPG-based locomotion framework and the deep reinforcement learning approach. To maintain conciseness, only the essential components required to interpret the results are discussed. As the majority of the theoretical background is outlined in [1], we only provide the design choices we explicitly made.

A. Pure CPG-Based Locomotion

We begin with pure CPG-based locomotion, for which multiple gait patterns were implemented, namely trot, pace, bound, and walk. While stable locomotion was more readily achieved for certain gaits (trot and pace) than for others (bound and walk), the design choices underlying all considered gaits are presented for completeness of this specific section.

1) *Gait-Specific Coupling Matrices*: At the outset, we present the coupling matrices for each gait, denoted by ϕ_{gait} , which define the inter-leg phase relationships and, consequently, the gait identity. As a representative example, the coupling matrix corresponding to the trot gait attains the form

$$\phi_{\text{trot}} = \begin{bmatrix} 0 & \pi & \pi & 0 \\ -\pi & 0 & 0 & -\pi \\ -\pi & 0 & 0 & -\pi \\ 0 & \pi & \pi & 0 \end{bmatrix},$$

as this coupling structure induces a trot gait by synchronizing diagonal leg pairs while enforcing a phase offset of π between the two pairs. Specifically, the front-right (FR) and rear-left (RL) legs, as well as the front-left (FL) and rear-right (RR) legs, oscillate in phase, whereas the two diagonal pairs remain in antiphase. In contrast, this matrix for the pace gait reads

$$\phi_{\text{pace}} = \begin{bmatrix} 0 & \pi & 0 & \pi \\ -\pi & 0 & -\pi & 0 \\ 0 & \pi & 0 & \pi \\ -\pi & 0 & -\pi & 0 \end{bmatrix},$$

as this coupling structure enforces a pace gait by synchronizing the ipsilateral leg pairs while imposing a phase difference of π between the left and right sides of the robot. In particular, the front-right (FR) and rear-right (RR) legs oscillate in phase, as do the front-left (FL) and rear-left (RL) legs, while the two lateral pairs remain in antiphase with respect to one another. This results in the characteristic lateral support pattern of the pace gait. The coupling matrix for the bound gait is

$$\phi_{\text{bound}} = \begin{bmatrix} 0 & 0 & \pi & \pi \\ 0 & 0 & \pi & \pi \\ -\pi & -\pi & 0 & 0 \\ -\pi & -\pi & 0 & 0 \end{bmatrix},$$

because this coupling structure induces a bound gait by synchronizing the front leg pair and the rear leg pair independently, while enforcing a phase difference of π between the

front and rear legs. Specifically, the front-right (FR) and front-left (FL) legs oscillate in phase, as do the rear-right (RR) and rear-left (RL) legs, whereas the front and rear pairs remain in antiphase with respect to each other. This coordination pattern yields the characteristic fore–hind symmetry of the bound gait. Finally, for the walk gait, we have the coupling matrix

$$\phi_{\text{walk}} = \begin{bmatrix} 0 & -\pi & \pi/2 & -\pi/2 \\ \pi & 0 & 3\pi/2 & \pi/2 \\ -\pi/2 & -3\pi/2 & 0 & -\pi \\ \pi/2 & -\pi/2 & \pi & 0 \end{bmatrix},$$

since this coupling structure produces a walk gait by enforcing sequential phase offsets between individual legs, resulting in a cyclic and asymmetric coordination pattern. Unlike the trot, pace, and bound gaits, the walk gait does not exhibit pairwise synchronization; instead, each leg oscillates with a distinct phase shift relative to the others, leading to a staggered footfall sequence. This phase progression ensures that at least three legs remain in contact with the ground at most times, yielding the characteristic support pattern for the walk gait.

2) *Hyperparameters*: Next, we consider the central pattern generator hyperparameters, which govern the temporal and geometric properties of the resulting locomotion. We outline the role of each parameter and discuss its influence on the observed gait behavior. The major tuning parameters are, as proposed in [1], the swing and stance frequencies, denoted by ω_{swing} and ω_{stance} , respectively. Yet, the more general set of tuning parameters is different. Specifically, one can extend the problem to the following hyperparameters:

- **Swing frequency**, ω_{swing} , controlling how fast the leg moves through the air (and therefore, how long it stays in the air), affecting overall speed, duty cycle, and stability.
- **Stance frequency**, ω_{stance} , determining how long the foot stays on the ground, which inherently influences support time and overall body stability of the quadruped.
- **Robot nominal height**, h , shifting the baseline foot trajectory, and influences stability and leg workspace.
- **Desired step length**, d_{step} , which directly scales forward stride, and affects body velocity and tracking realism.
- **Ground clearance**, g_c , determining maximum foot-lift during swing; too high causes instability, too low dragging.
- **Ground penetration**, g_p , defining nominal stance compression; impacts shock absorption and contact robustness.
- **Intrinsic amplitude**, μ , responsible for setting the intrinsic oscillation radius. Furthermore, it additionally affects stride amplitude and foot-trajectory magnitude.
- **Coupling strength**, w_{ij} , which controls how strongly legs synchronize, and ensures correct phase patterns for each gait.

Overall, this handful of hyperparameters impacts the locomotion performance the most. Some affect timing (e.g., the frequencies), some geometry (e.g., ground penetration), others the overall gait morphology. Generally speaking, the frequencies are the main driving factors, therefore staying true to [1], yet all of the parameters contribute to the result, as outlined above. To be more specific, all of the above parameters were explored to an extent, yet we have settled on keeping the default values for most, only modifying the swing and stance frequencies in

order to achieve the desired gait pattern, while remaining as stable and physically reasonable, *i.e.*, realistic, as possible.

3) *State Mapping and Control*: The states produced by the Hopf-based CPG are the the oscillator amplitude r_i , its phase θ_i , and their derivatives. We denote the states simply by the $(r, \theta, \dot{r}, \dot{\theta})$ quadruplet. They are mapped to desired Cartesian foot trajectories in the leg frame, following the formulation from [1]. Specifically speaking, the phase of each oscillator generates periodic foot motions in the sagittal plane, while the intrinsic amplitude μ and geometric parameters scale the resulting trajectories. The forward foot position is modulated by the desired step length and the oscillator phase, whereas the vertical component is shaped differently during swing and stance to enforce ground clearance and nominal ground penetration, respectively. This mapping yields smooth, continuous foot trajectories that are consistent with the prescribed gait timing and phase relationships. The resulting desired foot positions are converted to joint targets using inverse kinematics provided by the simulation framework. To track these trajectories, low-level joint torques are computed using a joint-space proportional-derivative (PD) controller. In addition, a Cartesian PD controller acting in task space can be incorporated to improve tracking performance by directly regulating foot position and velocity errors. The Cartesian and joint-space torque contributions are combined additively and applied at each control timestep, resulting in stable execution of the desired CPG-generated locomotion patterns.

4) *Locomotion Metrics*: To gain better insight into the performance, we follow the assignment by introducing some essential metrics we shall quantify in Section III. We consider:

- **Body velocity**, v_{body} , which comes as the first and most intuitive performance metric. Specifically, we evaluate the highest and lowest body velocity achieved during a simulation run of a given gait. The body velocity is obtained from the linear velocity of the robot base and is evaluated along the forward direction after discarding the initial transient.
- **Duty cycle**, D , which characterizes the temporal structure of the gait by quantifying the fraction of a stride period during which a given leg remains in contact with the ground. Mathematically speaking, it is defined in the form

$$D = \frac{T_{\text{stance}}}{T_{\text{stride}}},$$

wherein T_{stance} denotes the time spent in the stance phase and T_{stride} is the duration of one complete gait cycle for the same leg. In the considered CPG formulation, the stance and swing phases are encoded directly by the phase variable θ_i of each oscillator, with the swing phase corresponding to the range $0 \leq \theta_i \leq \pi$ and the stance phase to $\pi < \theta_i \leq 2\pi$. Consequently, the stance time over a stride can be obtained by accumulating the time intervals for which $\theta_i > \pi$, while the stride duration is given by the time elapsed between two consecutive phase wrap events. Since the simulation is executed in discrete time, both quantities are computed by counting up the number of timesteps belonging to the respective phases and multiplying by the simulation timestep Δt , yielding a discrete-time approximation of the duty cycle.

- **Stride duration**, T_{stride} , which is defined as the time required to complete one full gait cycle of a leg, *i.e.*,

$$T_{\text{stride}} = T_{\text{stance}} + T_{\text{swing}},$$

where the stance and swing durations, denoted by T_{stance} and T_{swing} , correspond to the time spent in ground contact and in the air, respectively. In simulation, this value is computed from the phase evolution of the CPG oscillators. A stride is defined as one full 2π -cycle of the phase variable, and stride boundaries are detected from phase wrap events. The stride duration is obtained as the time difference between two consecutive wraps and is computed in discrete time as the number of timesteps per cycle multiplied by the simulation timestep Δt . Stance and swing durations are obtained by accumulating timesteps belonging to the respective phase intervals within each stride of the quadruped.

- **Cost of transport**, CoT, which is a dimensionless metric used to quantify the energy efficiency of robot (or animal) locomotion. It can be defined in the form

$$\text{CoT} = \frac{E}{mgd},$$

where E is the total energy expended during locomotion, m is the robot (or animal) mass, $g \approx 9.81$ is the gravitational acceleration, and d is the total distance traveled by the center of mass. To evaluate this metric for our scenario, we first need to get from instantaneous power to energy. Here, the instantaneous mechanical power of joint i reads

$$P_i(t) = \tau_i(t)\dot{q}_i(t),$$

where $\tau_i(t)$ is the joint torque and $\dot{q}_i(t)$ is the joint angular velocity. The total energy is then computed as

$$E = \int_{t_0}^{t_1} \sum_{i=1}^{12} |\tau_i(t)\dot{q}_i(t)| dt,$$

wherein the absolute value is used to account for all actuator effort, including braking phases where the instantaneous mechanical power may be negative, yielding a conservative estimate of the total energy expenditure. Yet, since the model is inherently simulated in discrete time, we approximate the integral using a Riemann sum, *i.e.*,

$$\int_{t_0}^{t_1} f(t) dt \approx \sum_{k=0}^{N-1} f(t_k)\Delta t,$$

where k is a discrete timestep index and N is the number of simulation timesteps, over which energy is accumulated.

Overall, the metrics provide a compact yet informative characterization of the resulting locomotion, capturing kinematic performance, temporal gait structure, and energetic efficiency.

B. Deep Reinforcement Learning

We next consider locomotion controllers based on deep reinforcement learning, in which the control policies are learned directly through interaction with a physics-based simulation environment. In contrast to the CPG-based approach, which relies on explicit gait parametrization and phase coordination, DRL enables the discovery of locomotion strategies by virtue

of optimizing task-specific reward functions. In this section, we describe the formulation of the DRL problem, the design choices underlying the Markov decision process (MDP), and the training procedure used to obtain the resulting policies.

1) *Problem Formulation*: Before detailing the specific design choices, we first outline the learning problems addressed in this work in order to establish a clear baseline for the discussion that follows. To improve clarity and readability, both the problem formulation and the subsequent methodological components are organized according to the task under consideration. In particular, we distinguish between the following two problem settings for deep reinforcement learning:

- **Forward locomotion**, where the robot must walk (or run) forward on flat terrain while tracking a desired velocity.
- **Task-specific locomotion**, where the robot must navigate some non-trivial terrain, *i.e.*, either gaps, stairs, or slopes.

Both problem settings can, in principle, be addressed using joint-space PD control, Cartesian PD control, or by modulating CPG hyperparameters. Here, we adopt the latter approach, as it naturally extends the CPG-based framework introduced in the first part and provides a structured yet flexible control interface for learning. Specifically speaking, we have chosen to train the pace and trot gaits for forward locomotion and task-specific locomotion, respectively, in order to introduce variety to the problem. For the task-specific locomotion problem, we focus on traversal over slopes, as they introduce a continuous variation in terrain inclination without discontinuities.

2) *Observation Space*: We begin the detailed description of the design choices underlying the Markov decision process by specifying the observation space. As discussed in the previous subsection, the formulation is structured according to the two considered tasks. Hence, we distinguish between the following observation spaces for deep reinforcement learning:

- **Forward locomotion**, `DEFAULT`, which we extend beyond the initial baseline in order to accommodate CPG-based deep reinforcement learning. In addition to joint angles and joint velocities, the observation includes the base orientation quaternion as well as the internal states of the CPG, namely the oscillator amplitudes r and phases θ . The phase information is represented using sine and cosine components to avoid discontinuities at phase wrap-around. This extension is essential, as the policy does not act directly in joint space but instead modulates the parameters of an internal nonlinear dynamical system whose time evolution cannot be inferred from robot proprioception alone. Explicitly including the CPG state restores the Markov property of the resulting decision process and enables phase-aware modulation of the gait. The `DEFAULT` observation space is intentionally kept compact for this task, since flat terrain forward locomotion does not require explicit information about global position, terrain geometry, or base linear velocity, allowing the policy to focus on learning stable and coherent gait generation.
- **Task-specific locomotion**, `LR_COURSE_OBS`, where the observation space is extended even further to capture additional state information required for locomotion over uneven terrain. In addition to all quantities included in `DEFAULT`, the observation space includes the base linear and angular

velocities, the base height, and a compact representation of the local terrain inclination. These quantities provide direct feedback on forward progress, body stability, and contact quality, which become critical when traversing slopes. The terrain inclination information is encoded using the body pitch, together with its sine and cosine components, yielding a bounded, continuous representation that allows the policy to distinguish between uphill, flat, and downhill regions. By augmenting the observation space with these task-relevant signals, the policy can adapt its posture and gait timing, while retaining the structured CPG-based control paradigm.

Overall, the observation spaces are designed to balance expressiveness with parsimony, exposing only task-relevant quantities that are either directly measurable on hardware or can be reliably estimated using standard proprioceptive sensing. For both control policies, the observation is limited to low-level proprioceptive signals and internal controller states, avoiding reliance on global position information or privileged terrain knowledge, which is a technique used in teacher-student reinforcement learning. This design choice encourages the policy to exploit structured control through the CPG while maintaining a formulation that is compatible with realistic sensing assumptions. More advanced considerations, *e.g.*, sensor noise, dynamics randomization, and environment perturbations, are discussed separately as part of the environment and training details, in accordance with the assignment guidelines.

3) Action Space: We next specify the action space of the Markov decision process. In contrast to the observation space mentioned in the previous subsection, the action representation is shared across both considered tasks. In all cases, the policy does not act directly in joint or Cartesian space, but instead modulates the parameters of a central pattern generator (CPG) that produces structured locomotion patterns. The action is an eight-dimensional continuous vector, where each component lies in a normalized range and is linearly scaled to physically meaningful CPG parameters. Specifically, the policy outputs per-leg modulations of the intrinsic oscillator frequencies and amplitude setpoints. These parameters are updated at each decision step and provided as inputs to the Hopf-based CPG, which then evolves its internal dynamics and generates desired foot trajectories. The resulting trajectories are tracked using inverse kinematics and PD control. Importantly, while the policy continuously modulates the CPG parameters online, the inter-leg coupling structure of the CPG remains fixed. As a result, the learned policy adapts the execution of a predefined gait pattern rather than altering the underlying gait topology. This hierarchical formulation introduces a strong inductive bias toward coordinated and dynamically consistent locomotion, reducing the dimensionality of the reinforcement learning problem and improving stability compared to direct low-level control. At the same time, the continuous modulation of oscillator parameters enables sufficient flexibility to adapt gait timing and step characteristics to different requirements.

4) Reward Design: We now describe the reward design underlying the reinforcement learning formulation. Consistent with the structure of the preceding subsections, the reward functions are defined according to the two considered tasks. While both tasks share common objectives related to stable

and efficient locomotion, their reward formulations differ in how task-specific behaviors are incentivized. Accordingly, we distinguish between the following reward functions:

- **Forward locomotion**, wherein the reward function is designed to encourage stable forward locomotion at a desired speed, while penalizing unstable, inefficient, or degenerate behaviors. The dominant component of the reward promotes tracking of a desired forward base velocity along the body x -axis. Instead of using a purely linear error term, we employ smooth exponential shaping of the form

$$\mathcal{Q}_{\text{velocity}} = \exp\left(-\frac{(v_x - v_x^{\text{ref}})^2}{\sigma_v^2}\right),$$

where v_x denotes the measured base linear velocity in the forward direction, v_x^{ref} is a task-dependent reference velocity, and σ_v determines the width of the tolerance region. This formulation yields denser reward gradients near the target velocity while avoiding overly harsh penalties for transient deviations, a fact which is particularly important during early training. To ensure directional consistency, a penalty on the base yaw angle is included. It reads

$$\mathcal{Q}_{\text{yaw}} = -|\psi|,$$

where ψ denotes the yaw component of the base orientation. This term actively discourages excessive heading deviations that would otherwise allow the policy to accumulate forward velocity while rotating or "zig-zagging," a common failure mode in unconstrained locomotion tasks. In addition, lateral drift is penalized through a regularization term on the base position in the lateral direction. Specifically,

$$\mathcal{Q}_{\text{lateral}} = -|y|,$$

which is used to suppress sideways motion and encourage the learned gait to remain aligned with the forward direction. While small lateral deviations are unavoidable during locomotion, explicitly penalizing drift improves repeatability and stabilizes training. Energy efficiency is encouraged via a penalty on the mechanical work produced by the actuators. It is computed in similar fashion to the one in the CoT metric, and serves as a proxy for electrical energy consumption and discourages unnecessarily large torques or high-frequency oscillations. Importantly, this term is weighted such that it regularizes the solution without heavily overpowering the primary velocity objective. Penultimately, a posture regularization term penalizes deviations of the base orientation,

$$\mathcal{Q}_{\text{posture}} = -\|q_{\text{base}} - q_{\text{up}}\|,$$

where q_{base} denotes the base orientation and q_{up} corresponds to the nominal upright pose. This term suppresses solutions that achieve forward velocity at the cost of excessive roll or pitch, which would be undesirable both for stability and for transfer to hardware. Finally, the per-step reward is obtained as a simple weighted sum of the individual components and is clipped to be non-negative, *i.e.*,

$$\varrho \leftarrow \max\{0, \varrho\},$$

to prevent large negative returns from dominating the learning signal during early stages of training. In combination,

these terms guide the learning process toward smooth, stable, and energetically reasonable forward locomotion while remaining compatible with the CPG-based control structure.

- **Task-specific locomotion**, wherein the reward function is designed to promote robust forward traversal over uneven terrain while maintaining stability, controlled speed, and energy efficiency. In contrast to the forward locomotion task, where velocity tracking constitutes the primary objective, the dominant component here promotes forward progress along the course. This is implemented using an incremental reward

$$\mathcal{Q}_{\text{progress}} = \Delta x,$$

where Δx denotes the forward displacement of the robot base between consecutive decision steps, reasonably bounded to prevent exploitation of simulator artifacts. This formulation directly rewards meaningful advancement along the terrain without relying on absolute position information. To retain a notion of speed regulation, a secondary velocity tracking term is included, which encourages locomotion within a desired speed range without enforcing strict velocity tracking. This term has the same structure as in the first case, and helps prevent degenerate solutions such as excessively slow motion on flat sections or uncontrolled acceleration during downhill traversal. To encourage sustained locomotion and discourage premature episode termination, a small, constant alive bonus is added at each decision step, *i.e.*,

$$\mathcal{Q}_{\text{alive}} = c_{\text{alive}},$$

which provides a constant positive reward as long as the robot remains upright and the episode has not terminated. This term stabilizes learning by favoring behaviors that maintain balance and avoid falling, without encoding any overly specific task objective. Stable body posture is enforced through structured regularization of the base orientation. Lateral tipping is penalized via a roll-dependent term, while pitch deviations are shaped according to the local terrain inclination. Specifically, we have

$$\mathcal{Q}_{\text{posture}} = -(|\varphi| + |\xi - \xi^{\text{ref}}|),$$

where φ and ξ denote the roll and pitch angles of the base, respectively, and ξ^{ref} represents a terrain-dependent desired pitch. This formulation allows the policy to lean slightly into uphill or downhill sections while discouraging excessive body rotations that would compromise stability. Directional consistency is further encouraged by penalizing yaw deviations and lateral drift, *i.e.*,

$$\mathcal{Q}_{\text{yaw}} = -|\psi|, \quad \mathcal{Q}_{\text{lateral}} = -|y|,$$

which suppress heading oscillations and sideways motion that frequently arise when negotiating inclined terrain. Energy efficiency is encouraged in the same way as in the previous case. It discourages unnecessarily aggressive motions while preserving sufficient actuation authority to overcome slopes. Finally, additional task-specific shaping terms are introduced to address slope-induced failure modes. Backward sliding is penalized whenever the forward displacement becomes negative, while excessive forward speed during downhill traversal is discouraged by penalizing velocities

exceeding a nominal margin. These terms explicitly account for the asymmetric dynamics of uphill and downhill locomotion and improve training stability. The per-step reward is obtained as a weighted sum of the individual components and is clipped to be non-negative,

$$\varrho \leftarrow \max\{0, \varrho\},$$

to prevent large negative returns from dominating the learning signal during early stages of training. In combination, this reward formulation encourages steady and controlled traversal of uneven terrain while preserving the structured, phase-coherent locomotion induced by the underlying CPG.

To summarize, the reward functions are designed as structured combinations of interpretable components that directly reflect the objectives and constraints of each task. For both cases, the dominant reward terms represent task completion, while additional shaping and regularization terms penalize instability, inefficient actuation, and degenerate behaviors commonly observed in unconstrained locomotion learning. The forward locomotion reward emphasizes velocity tracking on flat terrain, whereas the task-specific formulation replaces strict velocity objectives with progress-based motivation and terrain-aware shaping to handle asymmetric uphill and downhill dynamics. Importantly, all reward components are defined using quantities that are either directly measurable or reliably estimable from on-board sensing, and no privileged information about environment geometry is used. This design ensures that the learning problem remains well-posed, physically grounded, and compatible with the hierarchical CPG-based control structure, while providing sufficiently dense feedback to enable stable and efficient control policy learning.

5) *Learning Algorithm*: Moving forward, we detail the deep reinforcement learning approach used to train the locomotion policies. Specifically, we discuss the choice of learning algorithm, highlight relevant differences between alternative algorithms and their implications for locomotion performance, and outline the policy network architecture together with the most influential hyperparameters. Thus:

- **Algorithm choice**, where policy learning is performed using Proximal Policy Optimization (PPO) as the primary reinforcement learning algorithm. PPO is an on-policy, policy-gradient method designed to provide stable learning through constrained policy updates, implemented via a clipped surrogate objective. This property is particularly important in legged locomotion tasks, where abrupt policy changes can easily destabilize contact dynamics and disrupt coordinated patterns. In our setting, the policy outputs continuous modulations of CPG parameters rather than low-level torques, making smooth and incremental policy updates especially desirable in order to preserve phase coherence and inter-leg coordination. We additionally considered Soft Actor-Critic (SAC), an off-policy algorithm that augments the standard reinforcement learning objective with an entropy maximization term to encourage exploration. While SAC is generally more sample-efficient, its higher sensitivity to reward scaling and a more aggressive exploration behavior were observed to result in less stable gait execution during early training,

particularly in contact-rich scenarios. In contrast, PPO exhibited more consistent convergence and produced smoother, more reliable locomotion policies across both tasks. Based on these considerations, PPO was selected as the primary learning algorithm for all reported results. [2]–[6]

- **Network architecture**, wherein both the policy and value functions are parameterized using feedforward neural networks with a multilayer perceptron (MLP) structure. More specifically, the policy network maps the observation vector to a continuous action distribution over the CPG modulation parameters, while a separate value network estimates the state value required for policy gradient optimization. In all experiments, both networks employ identical architectures consisting of two fully connected hidden layers with nonlinear activation functions. This choice was found to be sufficient given the moderate dimensionality of the observation space and the low-dimensional, structured action representation induced by the CPG-based control. The use of a relatively shallow MLP architecture is further motivated by the hierarchical formulation of the control problem. Since the policy operates on high-level CPG parameters rather than directly commanding joint torques, the mapping from observations to actions is smoother than in direct low-level control settings. Therefore, expressive but lightweight networks are sufficient to capture the relevant control relationships, while avoiding unnecessary model complexity that could hinder learning stability or increase sample requirements. [7], [8]
- **Hyperparameters**, which strongly influence learning stability and policy behavior in deep reinforcement learning. In this work, we remain conservative and rely primarily on the default hyperparameter settings provided by the Stable-Baselines3 implementation of PPO. During preliminary experiments, key hyperparameters were varied within reasonable ranges to assess their qualitative impact on training stability and locomotion behavior. Based on these observations, the default configuration was kept, as it consistently yielded stable learning and reliable performance without requiring extensive tuning. Among the most influential hyperparameters are the learning rate, the rollout horizon, and the discount factor. The learning rate controls the magnitude of policy updates and directly affects convergence stability, while the rollout length determines the temporal context used for advantage estimation and must be sufficiently long to capture the periodic structure of the CPG-induced gait. The discount factor governs the balance between short-term stability and long-term task objectives, encouraging sustained locomotion over transient reward exploitation. In addition, PPO-specific parameters such as the clipping range and the generalized advantage estimation (GAE) coefficient limit overly aggressive policy updates while preserving an informative learning signal. Together, these hyperparameters provide a stable optimization regime for contact-rich quadruped locomotion learning. [2]–[4], [7]–[9]

Collectively, the learning algorithm is selected and configured to prioritize not only stability but also robustness in a contact-rich locomotion setting. The specific combination of a conservative on-policy optimization method, a lightweight MLP-based policy and value architecture, and empirically validated

hyperparameter defaults aligns well with the hierarchical CPG-based formulation. Together, these design choices enable reliable policy learning while avoiding unnecessary algorithmic complexity, ensuring that performance differences across tasks primarily arise from the task formulation and reward design rather than from aggressive optimization strategies.

6) *Training Procedure*: The training process is performed in simulation using the PyBullet physics engine with a fixed integration time step. The control policy operates at a lower frequency through action repetition, resulting in a consistent separation between physics simulation and decision-making. Episodes are terminated either when the robot is detected to have fallen, based on base height and orientation criteria, or when a time horizon is reached. For forward locomotion on flat terrain, a shorter episode length is used, while the task-specific locomotion on sloped terrain employs longer episodes to allow the robot sufficient time to traverse through both the uphill and downhill sections of the course. To improve robustness and prevent overfitting to idealized dynamics, moderate environment stochasticity is introduced during training. This includes additive observation noise as well as randomization of ground friction coefficients at reset. Terrain geometry is fixed within each task but differs across tasks, with flat ground used for forward locomotion and a structured sloped course used for task-specific training. Each task is trained independently using the same learning pipeline and network architecture, differing only in reward formulation, observation space, and episode duration. During evaluation, normalization statistics are frozen and environment noise is disabled to assess the learned policies under nominal conditions. As a final remark, we have found Google Colab to be the ideal resource for training.

III. RESULTS

Having successfully established the methodology of this work, we now present the results obtained throughout the project. For clarity and ease of navigation, this section is organized chronologically according to the progression of the assignment. We first report the results for pure CPG-based locomotion, followed by those obtained using deep reinforcement learning. To maintain focus on the principal findings, only the most relevant results are included in this section, while additional results are provided in the supplementary Appendix A.

A. Pure CPG-Based Locomotion

We begin by presenting the results obtained using pure CPG-based locomotion. In this subsection, we detail the selected CPG parameters and evaluate the resulting locomotion behavior for the slow trot gait. Specifically, we analyze foot position tracking performance, joint-space tracking accuracy, and key locomotion metrics, including body velocity, duty ratio, stance and swing durations, and cost of transport. An overview of the fast trot gait and additional gaits is provided in Appendix A.

1) *Selected Parameters*: As we discussed in the methodology section, the primary parameters influencing the resulting locomotion behavior are the stance and swing frequencies, as well as the Cartesian PD gains when Cartesian control is employed. Through manual tuning, we identified parameters that produced stable locomotion, which are summarized in the

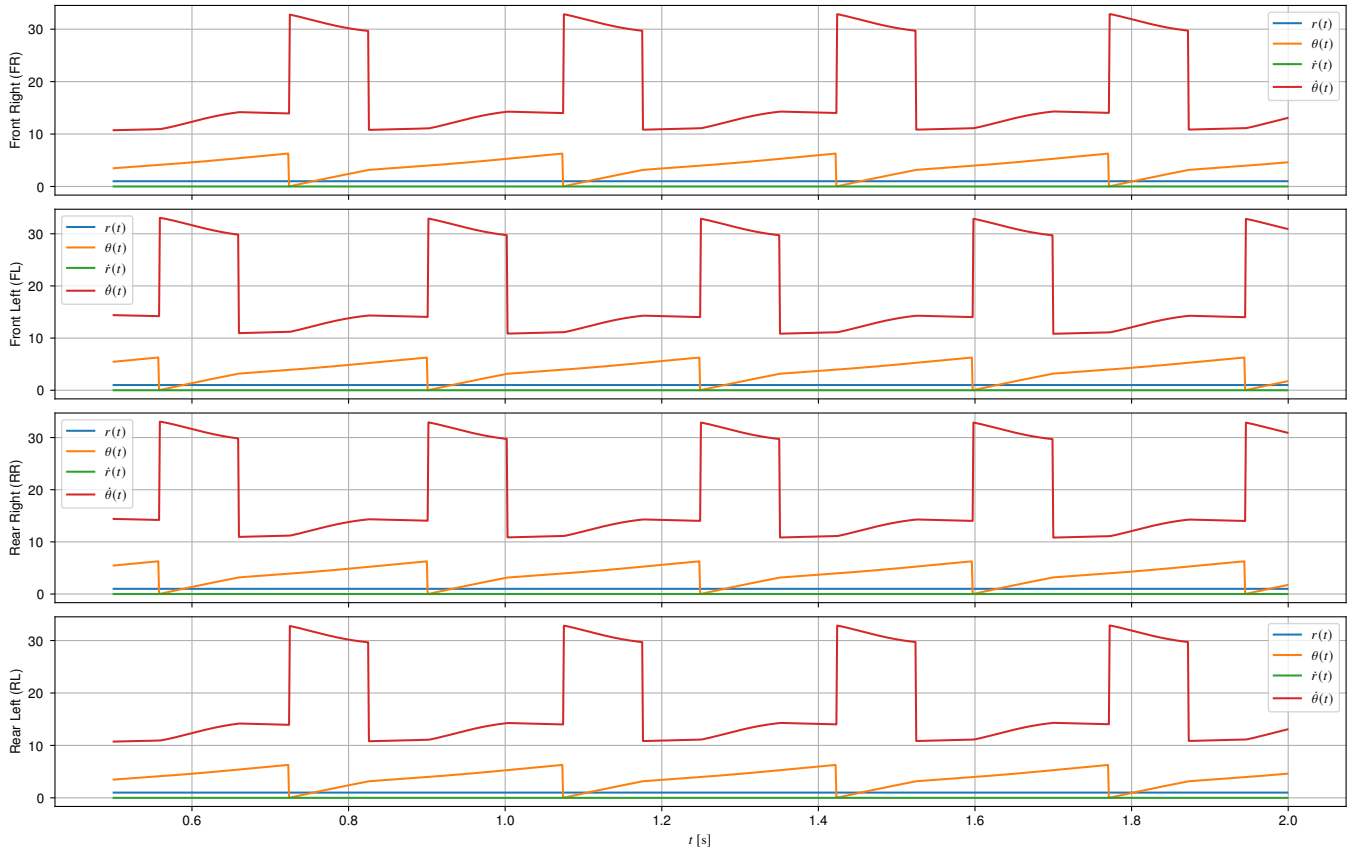


Figure 1: Time evolution of the CPG states. The slow trot gait is shown.

following table. It is worth noting that both the joint PD and Cartesian PD gains were kept at their provided defaults, *i.e.*,

```
# joint PD gains
kp=np.array([100,100,100])
kd=np.array([2,2,2])

# Cartesian PD gains
kpCartesian = np.diag([500]*3)
kdCartesian = np.diag([20]*3)
```

as we found them to yield satisfactory performance in combination with the selected frequencies. We therefore choose to list only the relevant stance and swing frequencies.

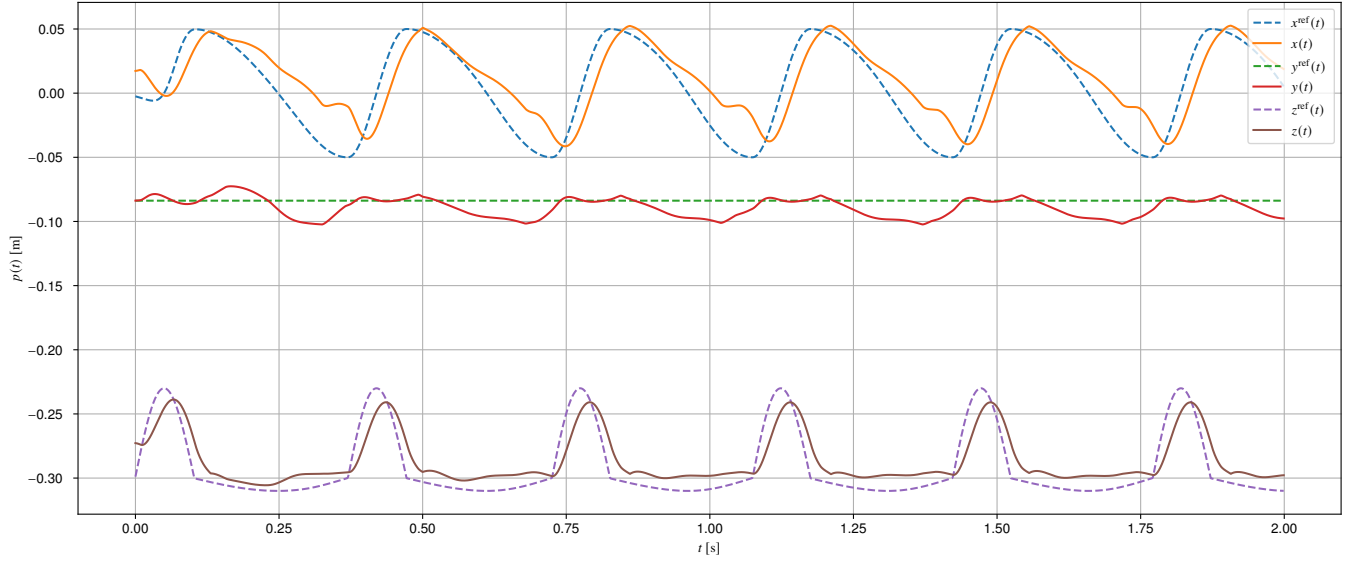
Parameter Name	Symbol	Value	Unit
Stance frequency	ω_{stance}	4π	[rad/s]
Swing frequency	ω_{swing}	10π	[rad/s]

Table 1: Selected parameters. The slow trot gait is shown.

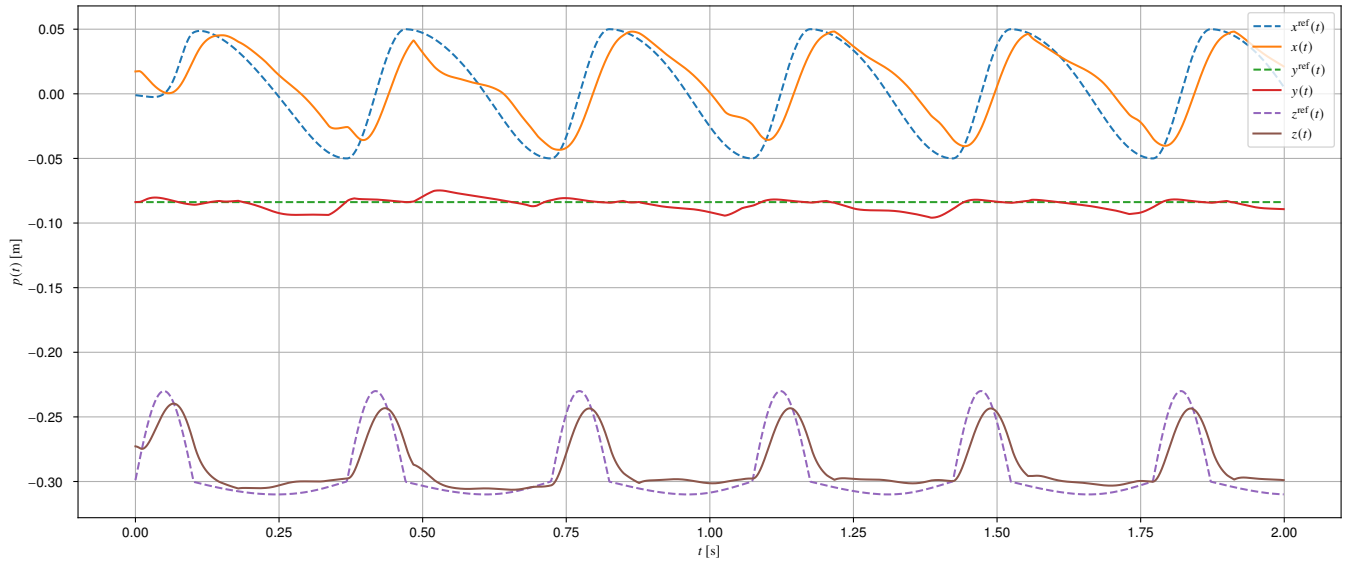
Using the selected stance and swing frequencies summarized in Table 1 together with the trot coupling matrix ϕ_{trot} , Figure 1 shows the resulting time evolution of the CPG state variables ($r, \theta, \dot{r}, \dot{\theta}$) for the slow trot gait. Following a short transient phase, the oscillator amplitudes converge to steady values for all four legs, indicating stable limit-cycle behavior under the chosen parameterization. The corresponding phase trajectories evolve periodically, with clear alternation between swing

and stance phases as reflected by the piecewise evolution of their derivatives. The imposed coupling structure yields the intended inter-limb coordination: the diagonal leg pairs remain phase-aligned, while the remaining leg pairs exhibit a phase offset. This phase relationship is consistently preserved over successive gait cycles and results in alternating diagonal support phases. Such coordination is characteristic of trotting locomotion and confirms that the selected frequencies, together with the coupling matrix, produce the desired gait pattern.

2) *Tracking Performance on Foot Positions:* We next evaluate the ability of the controller to track the desired Cartesian foot trajectories generated by the CPG. Fig. 2 compares the desired and measured foot positions for a representative leg during slow trot locomotion, using joint PD control alone and joint PD augmented with Cartesian PD control. The reference trajectories correspond to the CPG-mapped foot motion in the leg frame, as described in the methodology. When using joint PD control only (Fig. 2a), the overall foot motion follows the desired periodic pattern; however, noticeable tracking errors are present during phase transitions between swing and stance. These deviations come out as phase lag and reduced peak amplitudes. By contrast, augmenting the controller with Cartesian PD feedback (Fig. 2b) leads to visibly improved tracking performance, mainly in reducing some oscillation, specifically within the lateral direction. The foot trajectories are a closer match to the desired references across all Cartesian



(a) Joint PD without Cartesian PD.



(b) Joint PD with Cartesian PD.

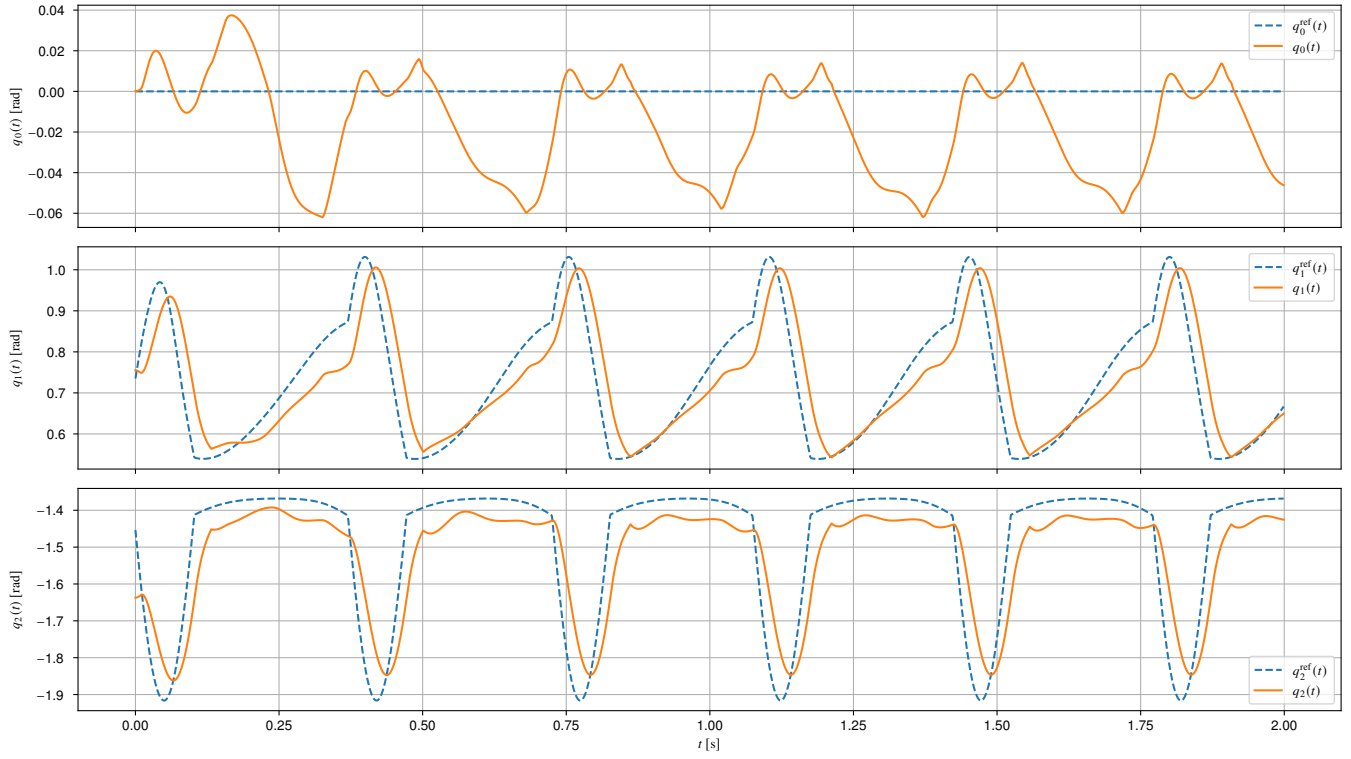
Figure 2: Tracking performance on foot positions. The slow trot gait is shown.

directions, with marginally reduced phase lag and improved amplitude consistency. This improvement arises from directly regulating task-space errors through the Jacobian transpose, allowing the controller to better compensate disturbances and kinematic coupling effects. Overall, these results demonstrate that Cartesian PD control enhances foot placement accuracy compared to joint PD control alone, even if the effect is minor due to the selected gains we have shown before.

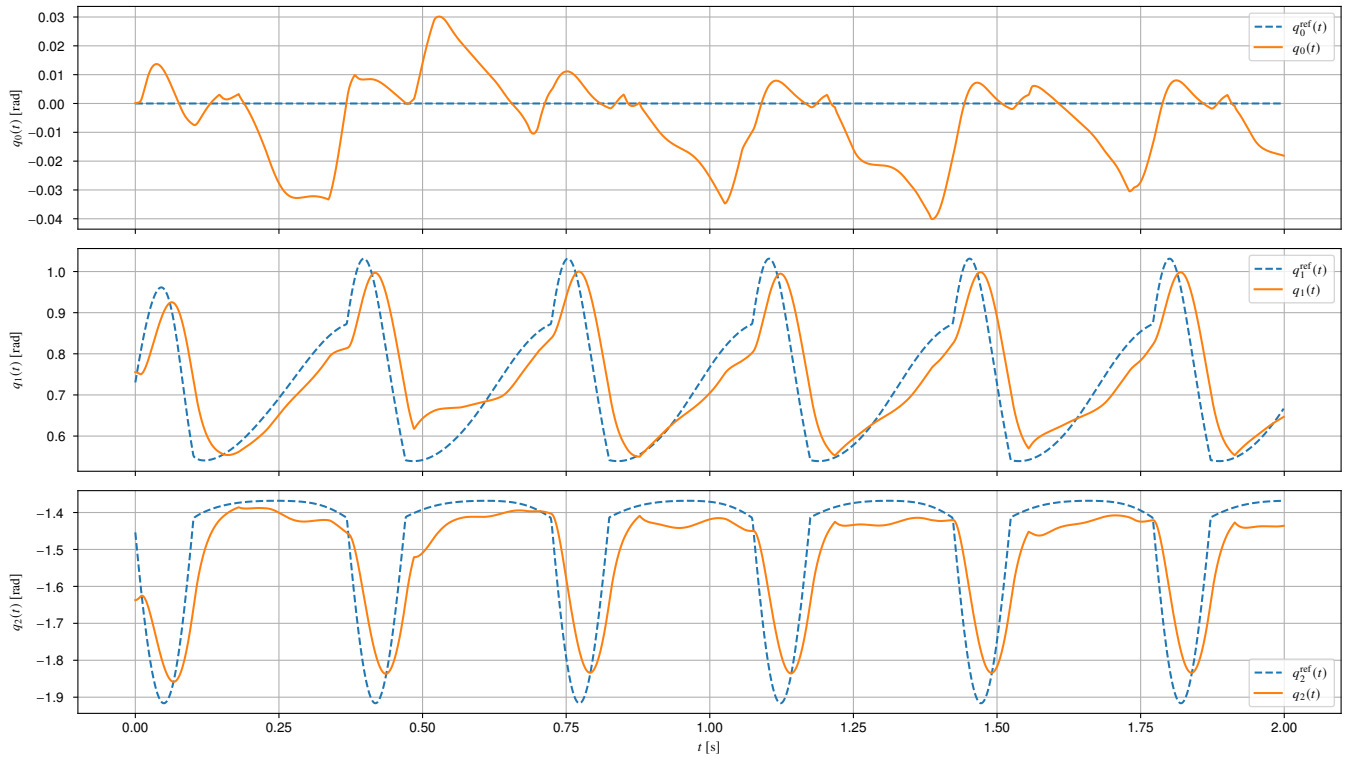
3) Tracking Performance on Joint Angles: We move on to tracking performance in joint space to assess how accurately the commanded joint trajectories are followed during slow trot locomotion. Figure 3 compares the desired and measured joint angles for a representative leg using joint PD control alone and joint PD augmented with Cartesian PD. When only

joint PD control is employed (Fig. 3a), the overall periodic structure of the reference trajectories is preserved. However, noticeable tracking errors again arise. Specifically, we can notice some unwanted oscillation during stance/swing change in the third joint, which Cartesian PD (Fig. 3b) tries to mitigate. Additionally, it shrinks the already small oscillations around the reference in the first joint. In summary, tracking performance is again improved only marginally; a fact which we again contribute to the gains chosen. On the other hand, we dare to pronounce the joint-level tracking sufficient, the same as the foot positions tracking, shown previously.

4) Locomotion Metrics: We conclude the analysis of the slow trot gait by reporting the locomotion metrics introduced in the methodology. Particularly speaking, we evaluate the minimum



(a) Joint PD without Cartesian PD.



(b) Joint PD with Cartesian PD.

Figure 3: Tracking performance on joint angles. The slow trot gait is shown.

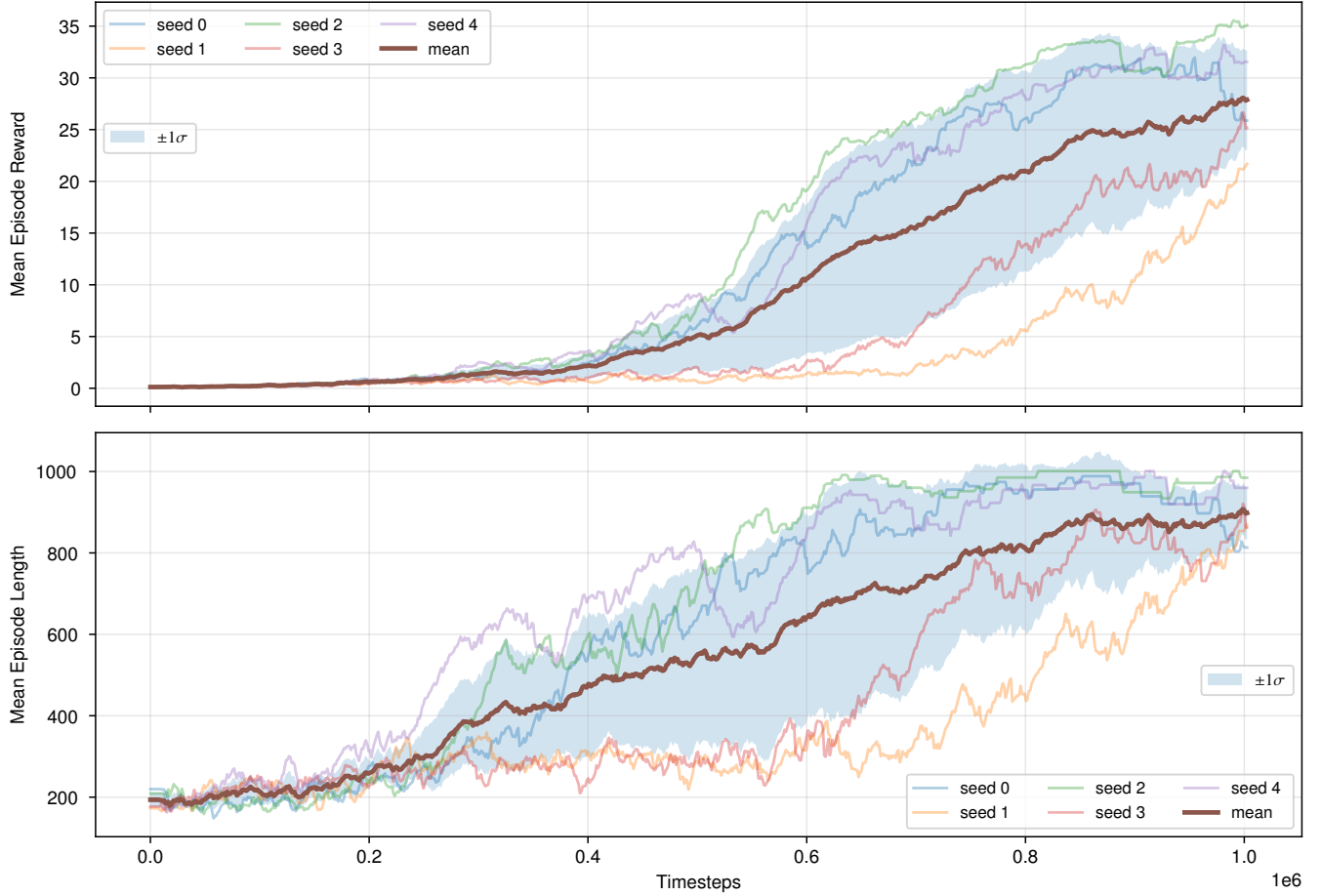


Figure 4: PPO training curves for forward locomotion.

and also maximum body velocities, the duty cycle, the stride duration, and the resulting cost of transport. The corresponding quantitative results are provided within Table 2 below.

Performance Metric	Symbol	Value	Unit
Minimum body velocity	$\min v_{\text{body}}$	0.16	[m/s]
Maximum body velocity	$\max v_{\text{body}}$	0.48	[m/s]
Duty cycle/ratio	D	0.71	[1]
Stance duration	T_{stance}	0.24	[s]
Swing duration	T_{swing}	0.10	[s]
Stride duration	T_{stride}	0.34	[s]
Cost of transport	CoT	1.10	[1]

Table 2: Performance metrics. The slow trot is shown.

All reported metrics were computed over a 10 s locomotion interval with Cartesian PD control enabled and therefore reflect steady-state gait behavior rather than transient effects. The relatively narrow range between the minimum and maximum forward velocities indicates stable progression with limited speed fluctuations. The observed duty ratio of $D = 0.71$ confirms that the gait is stance-dominated, which is characteristic of slow and conservative trotting and contributes to improved stability at the expense of speed. This behavior is consistent with the measured stance and swing durations, where the stance phase accounts for a significantly larger portion of the

stride. The resulting stride duration of 0.34 s further reflects a low-frequency gait pattern, aligning with the selected CPG parameters. The measured cost of transport of $\text{CoT} = 1.10$ lies within the expected range for CPG-driven quadruped locomotion without explicit energetic optimization. While higher than values reported for highly optimized legged systems, this CoT indicates a reasonable energetic efficiency given the simplicity of the control architecture and the emphasis on stability.

B. Deep Reinforcement Learning

We now present the results obtained using deep reinforcement learning-based locomotion controllers. In contrast to the pure CPG framework discussed previously, these controllers are learned through interaction with the simulation environment and exploit a hierarchical control structure in which the policy modulates the parameters of an underlying central pattern generator. This formulation preserves the rhythmic structure and coordination properties of CPG-based locomotion while enabling task-dependent adaptation through learning. The results are organized according to the learning problems introduced in the methodology. First, we evaluate a forward locomotion policy trained to track a desired velocity on flat terrain. This setting serves as a baseline to assess learning stability, convergence behavior, and the quality of the resulting

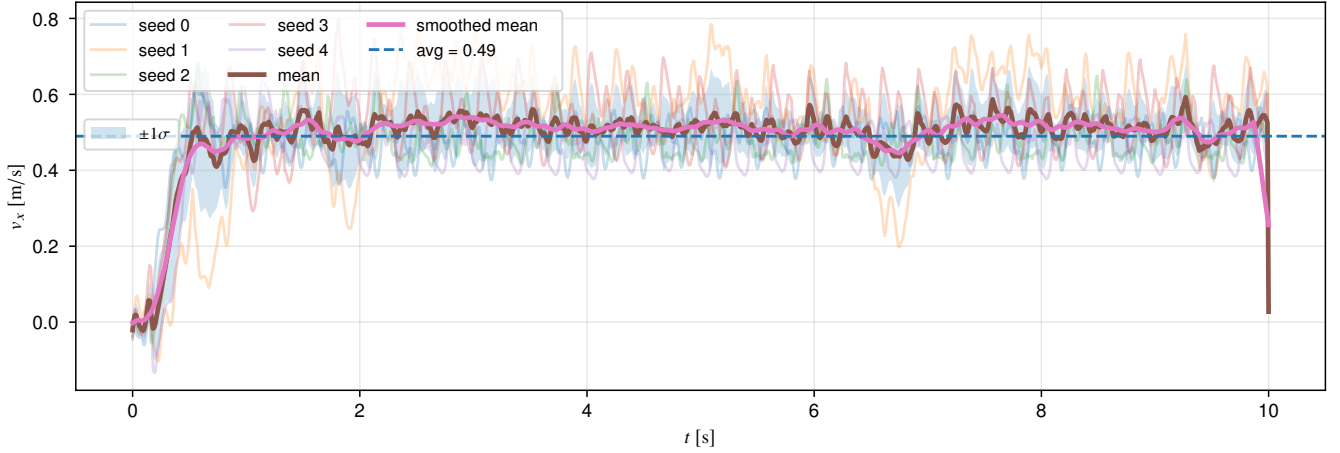


Figure 5: Base forward velocity for forward locomotion.

gait in the absence of terrain-induced disturbances. Second, we reduce strict velocity tracking and instead focus on task-specific locomotion over sloped terrain, where the policy must balance forward progress, stability, and energetic efficiency under continuously varying inclination. For both of the cases, we report training performance across multiple random seeds.

1) Forward Locomotion: First, let us begin with evaluating the forward locomotion task on flat terrain. For this setting, the episode length was kept at its default value of 10 s, and the policy was trained to track a target forward velocity of $v_x^{\text{ref}} = 0.5 \text{ m/s}$. The purpose of this experiment is twofold. First, to assess the overall training behavior and convergence properties of the learning process, and second, to evaluate the quality of the resulting locomotion in terms of forward velocity tracking. Figure 4 shows the PPO training curves for forward locomotion across five random seeds. The top subplot depicts the mean episode reward, while the bottom subplot depicts the corresponding mean episode length. Across all seeds, a clear learning trend can be observed. After an initial exploration phase with low rewards and short episode lengths, the policies gradually improve and converge toward sustained locomotion. The episode length is consistently increasing toward the maximum achievable value, indicating that the learned policies are able to maintain balance and avoid early termination. While some variability across seeds is present, particularly during the intermediate stages of training, the mean performance exhibits steady improvement, and the variance slightly decreases as training progresses. This behavior is characteristic for stable PPO training and suggests that the chosen observation space, action representation, and reward formulation lead to a well-posed reinforcement learning problem. To further analyze the resulting behavior, Figure 5 reports the base forward velocity during policy execution after training. Each curve corresponds to one random seed, with the mean and one-standard-deviation envelope shown for reference. Following a short transient at the beginning of the episode, all policies converge to a steady forward velocity close to the desired target. The averaged steady-state velocity is approximately 0.49 m/s, which very closely matches the reference velocity

of 0.5 m/s. The remaining deviation can be attributed to the stochastic nature of the learned policy, residual oscillations induced by the underlying central pattern generator dynamics, and somewhat conservative regularization terms in the reward function. Importantly, the forward velocity remains stable over the full episode duration, with almost no lateral drift, oscillatory divergence, or degradation in performance. Overall, these results demonstrate that the learned policy successfully acquires forward locomotion with accurate velocity tracking. When running the simulation, the robot exhibits smooth and stable forward locomotion visually. Qualitatively observing the resulting locomotion, it remains within realistic performance and could be ported to the real robot without having to worry about harming the hardware. As far as the locomotion metrics are concerned, they are shown within Table 3 below.

Performance Metric	Symbol	Value	Unit
Average body velocity	$\bar{v}_{\text{body}}$	0.49	[m/s]
Duty cycle/ratio	D	0.56 ± 0.07	[1]
Stance duration	T_{stance}	0.19 ± 0.02	[s]
Swing duration	T_{swing}	0.14 ± 0.02	[s]
Stride duration	T_{stride}	0.33 ± 0.04	[s]

Table 3: Performance metrics of forward locomotion.

Unfortunately, due to an error we have not been able to resolve, the CoT returns NaN (not a number), hence why we choose to not include it in the table. We understand it is an important metric, but due to time constraints, we were not able to resolve the error. Still, we think all of the components of this specific DRL problem have been well-designed. As a final remark, the reader should note Table 3 reflects the performance across all seeds that we ran for forward locomotion.

2) Task-Specific Locomotion: Now, let us turn to the evaluation of task-specific locomotion over uphill and downhill slopes. In contrast to the previous task, where strict velocity tracking constituted the primary objective, this setting emphasizes robust forward progress and stability under varying terrain inclination. As described in the methodology, the reward formulation prioritizes forward displacement while allowing

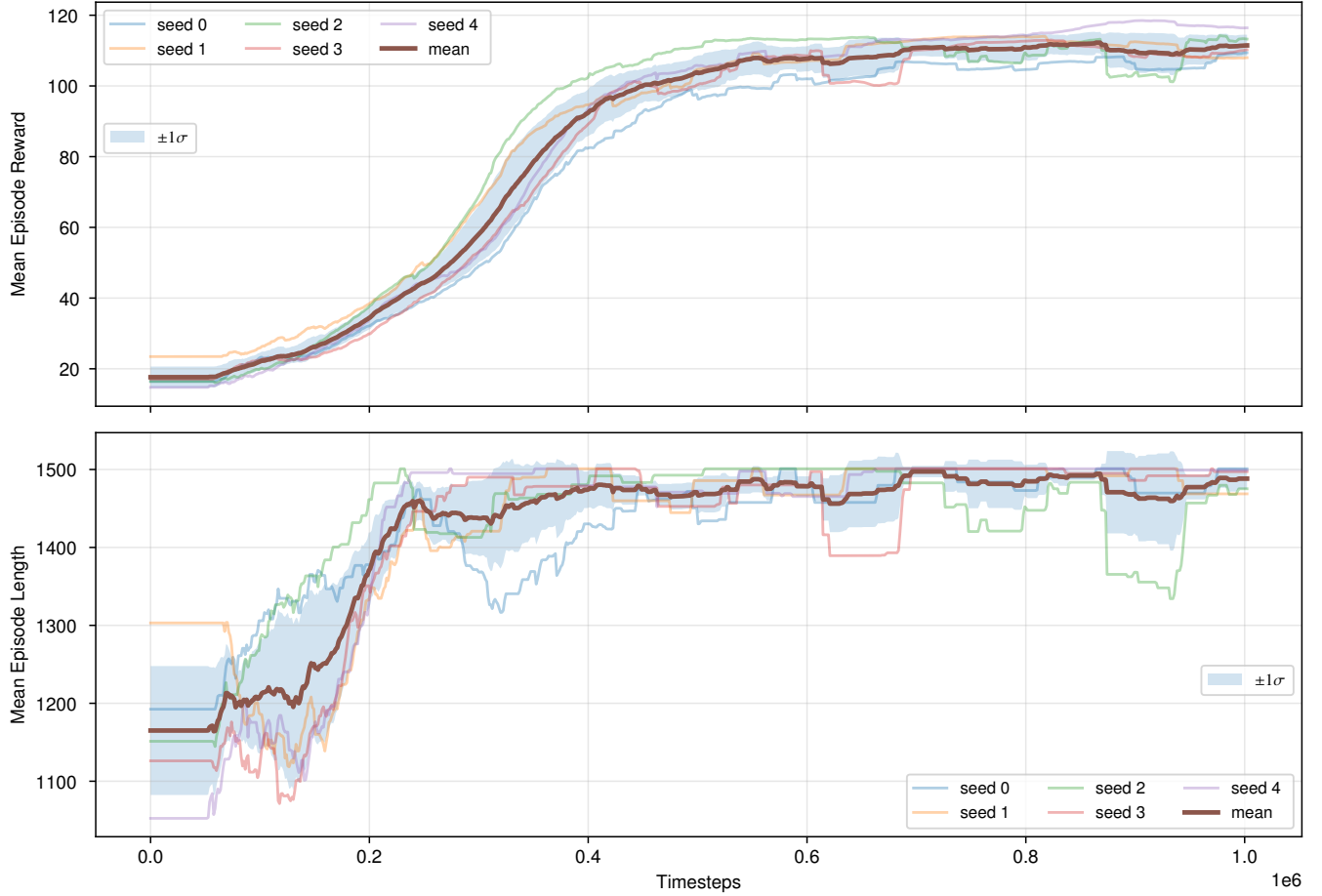


Figure 6: PPO training curves for task-specific locomotion.

the policy to adapt its speed, posture, and gait timing in response to uphill and downhill segments. Figure 6 presents the PPO training curves for task-specific locomotion across five random seeds. The top subplot shows the mean episode reward, while the bottom subplot details the corresponding mean episode length. Compared to forward locomotion, the learning progresses over a longer horizon and exhibits increased variability, reflecting the increased complexity of the task. Yet, it itself seems to be faster to learn than forward locomotion. Nevertheless, a consistent improvement trend is observed across all seeds. After an initial exploration phase, the episode length increases toward its maximum achievable value, indicating that the learned policies are able to traverse the sloped course without premature termination. The reward curves similarly converge to a high and stable plateau, suggesting that the policies successfully learn to balance forward progress, stability, and regularization terms from the reward function. While some drops in episode length occur for individual seeds, the mean performance remains stable, highlighting the robustness of the learning process. To analyze the resulting behavior in more detail, Figure 7 reports the base forward velocity as well as the base roll and pitch angles during policy execution. The forward velocity profile reveals a clear adaptation to terrain inclination. After an initial acceleration phase, the velocity

increases on uphill segments and decreases during downhill traversal, resulting in a mean forward velocity of approximately 0.38 m/s across seeds, while the required velocity was 0.4 m/s. This resulting behavior contrasts with the flat-terrain case, where the velocity converges tightly to a fixed reference, and instead reflects the progress-based objective of the task-specific reward. Importantly, despite substantial variation in instantaneous velocity, the motion remains smooth and free of abrupt oscillations. The base orientation plots further demonstrate stable locomotion under changing terrain conditions. The roll angle φ remains tightly regulated around zero for the majority of the episode, indicating effective lateral balance even during inclined traversal. The pitch angle ξ exhibits systematic variation that correlates with the slope profile, with the mean pitch remaining close to zero when averaged over the full trajectory. These trends confirm that the learned policy exploits body posture adaptation rather than relying purely on increased actuation effort to negotiate the terrain. Quantitative locomotion metrics aggregated across all seeds are summarized in Table 4. The duty ratio remains close to that observed in forward locomotion, indicating that the underlying gait structure is largely preserved despite task-specific adaptations. Slight increases in stride duration and stance time reflect a more conservative gait during challenging

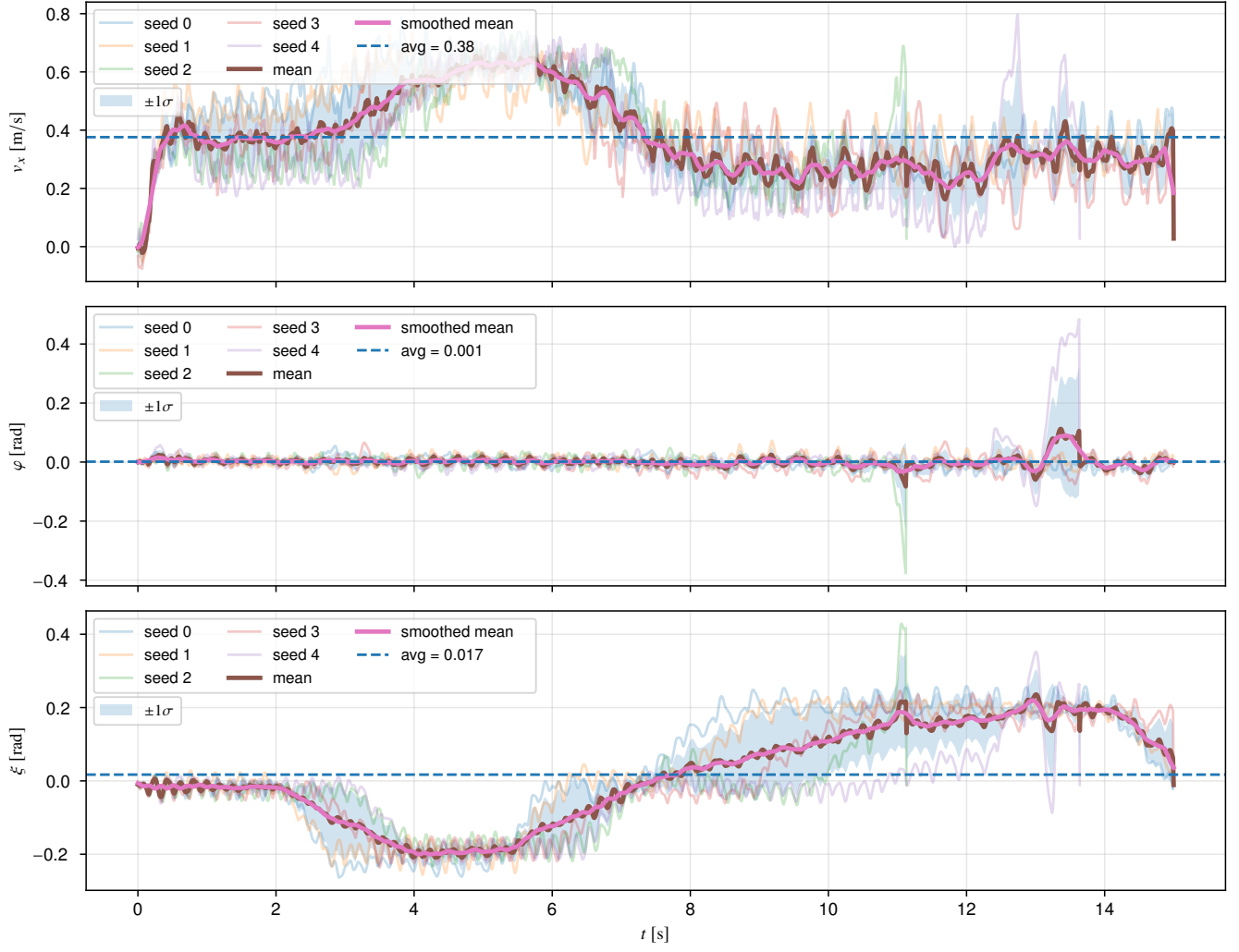


Figure 7: Base forward velocity, roll, and pitch for task-specific locomotion.

terrain sections, while swing duration remains comparable. As in the forward locomotion case, the cost of transport could not be reliably evaluated and resulted in undefined values due to numerical issues in our code. Consequently, it is not reported as a meaningful performance metric for this task.

Performance Metric	Symbol	Value	Unit
Average body velocity	$\bar{v}_{\text{body}}$	0.38	[m/s]
Duty cycle/ratio	D	0.59 ± 0.03	[1]
Stance duration	T_{stance}	0.23 ± 0.02	[s]
Swing duration	T_{swing}	0.16 ± 0.01	[s]
Stride duration	T_{stride}	0.39 ± 0.01	[s]

Table 4: Performance metrics of task-specific locomotion.

Overall, these task-specific results demonstrate that the learned policy successfully adapts CPG-based locomotion to nontrivial terrain without explicit terrain geometry information. By modulating gait timing, forward velocity, and body posture, the controller achieves stable and robust traversal of sloped terrain across multiple training runs. This confirms that the hierarchical CPG-based deep reinforcement learning formulation

generalizes beyond flat-terrain velocity tracking and enables task-specific adaptation while retaining smooth and physically plausible quadruped locomotion behavior.

IV. DISCUSSION

The results presented in the previous sections highlight several important aspects of quadruped robot locomotion control using both either pure CPG-based approaches, or CPG-augmented deep reinforcement learning. Rather than optimizing a single performance metric, the results reveal trade-offs between the core structure, adaptability, robustness, and interpretability that arise from different formulations. A key observation is the role of the central pattern generator as an effective inductive bias for both locomotion control paradigms. In the pure CPG setting, stable locomotion emerges from carefully tuned oscillator dynamics and kinematic mappings, but, unfortunately, remains fundamentally open-loop with respect to the environment. As a consequence, performance is highly sensitive to parameter selection, particularly swing and stance frequencies, step length, and ground clearance, among others. While these parameters can be tuned to achieve a wide range

of body velocities and duty ratios, the resulting controllers lack the ability to adapt online to disturbances or terrain variations beyond what is implicitly encoded in the gait design. By contrast, the CPG–DRL formulation preserves the rhythmic structure of the underlying gait while enabling task-dependent adaptation through learning. In the forward locomotion task, this manifests as rapid convergence to a stable pace gait that tracks the desired velocity with low steady-state error. The relatively small variance across random seeds and the consistent convergence of episode length suggest that the hierarchical structure simplifies the reinforcement learning problem by reducing the effective action space. Rather than learning locomotion from scratch, the policy learns to modulate an already stable limit cycle, which improves training stability and sample efficiency compared to fully unstructured control policies. The task-specific locomotion results further emphasize the benefits of this formulation. Unlike forward locomotion on flat terrain, sloped traversal requires continuous adaptation of velocity, posture, and gait timing. The learned policies exhibit systematic modulation of robot base pitch and forward velocity in response to terrain inclination, indicating that the agent exploits body-level strategies rather than relying solely on increased actuation effort. Importantly, this behavior emerges without explicit access to terrain geometry, suggesting that the observation space contains sufficient proprioceptive information for implicit terrain inference. At the same time, the increased variability observed during training highlights the inherent difficulty of the performed task and the sensitivity of reinforcement learning to stochastic exploration in contact-rich environments. Across both learning problems, the structure of the observation and action spaces plays a decisive role in the resulting performance. Including body orientation and joint velocities was proven essential for maintaining stability, while using CPG parameter modulation as the action space constrained the policy to physically plausible motions. This design choice improves smoothness and repeatability of the learned gaits, but also limits expressiveness: the policy cannot easily deviate from the nominal gait topology imposed by the generated pattern. In this sense, the approach trades generality for robustness and interpretability, which may be desirable in scenarios in which safety and predictability are prioritized. The reward formulation similarly reflects a balance between competing objectives. Velocity tracking rewards lead to precise steady-state behavior in the forward locomotion task, but become overly restrictive for uneven terrain, where progress-based rewards yield more adaptive strategies. The difficulty encountered when computing a meaningful cost of transport underscores an even broader challenge: energy efficiency is difficult to measure robustly in simulation and is highly sensitive to numerical integration, contact modeling, and episode termination conditions. From a robustness and transfer perspective, the presented results suggest both strengths and limitations. The learned policies are robust across random seeds and moderate terrain variations, indicating good generalization within the simulated environment. However, the reliance on idealized sensing, simplified actuator models, and fixed ground properties limits direct sim-to-real applicability. Bridging this “sim-to-real” difference would likely require

more sophistication in domain randomization, explicit noise injection, and other feedback mechanisms such as virtual model control or reflex-based corrections. Integrating such elements with the existing CPG–DRL framework represents a natural extension toward hardware deployment. Yet, we have not been able to do so, primarily due to time and human resource constraints. On the other hand, the provided results illustrate that combining structured locomotion primitives with deep reinforcement learning offers a practical compromise between classical control design and data-driven optimization. While the resulting controllers are not globally optimal in terms of speed or efficiency, they achieve stable, interpretable locomotion with relatively modest training effort.

V. CONCLUSION

In this project, we studied quadruped locomotion using both pure CPG-based control and a deep reinforcement learning framework. A hand-designed CPG-based controller was shown to produce stable gait patterns, but with limited adaptability to changing conditions. By extending the CPG structure with reinforcement learning, the resulting controllers achieved reliable velocity tracking on flat terrain and very robust traversal of sloped environments through task-dependent modulation of gait parameters. Overall, the provided results demonstrate that combining structured locomotion primitives with learning-based adaptation provides an effective and flexible approach to quadruped robot locomotion control.

REFERENCES

- [1] L. Gevers and A. Ijspeert, “Quadruped Locomotion with Central Pattern Generators and Deep Reinforcement Learning,” Mini-Project II Assignment, MICRO-507, Swiss Federal Institute of Technology, 2025.
- [2] J. Schulman, F. Wolski, P. Dhariwal et al., “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [3] OpenAI, “Proximal Policy Optimization,” 2018. [Online]. Available: <https://spinningup.openai.com/en/latest/algorithms/ppo.html>
- [4] O. Sigaud, “From Policy Gradient to Actor-Critic Methods, Proximal Policy Optimization (PPO),” Sorbonne Université, 2021. [Online]. Available: https://rl-vs.github.io/rlvs2021/class-material/pg/10_ppo.pdf
- [5] OpenAI, “Soft Actor-Critic,” 2018. [Online]. Available: <https://spinningup.openai.com/en/latest/algorithms/sac.html>
- [6] T. Haarnoja, A. Zhou, P. Abbeel and S. Levine, “Soft Actor-Critic, Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,” *arXiv preprint arXiv:1801.01290v2*, 2018.
- [7] A. Raffin et al., “Stable-Baselines3 Documentation,” [Online]. Available: <https://stable-baselines3.readthedocs.io/en/master/index.html>
- [8] D. Calin, “The Complete Practical Guide to PPO with Stable-Baselines3,” [Online]. Available: <https://www.reinforcementlearningpat h.com/the-complete-practical-guide-to-ppo-with-stable-baselines3/>
- [9] J. Schulman, P. Moritz, S. Levine et al., “High-Dimensional Continuous Control Using Generalized Advantage Estimation,” *arXiv preprint arXiv:1506.02438v6*, 2018.

APPENDIX A SUPPLEMENTARY RESULTS

To augment the results presented in this report, we provide a selected set of additional observations obtained throughout the project. These supplementary results offer further insight into the behavior of the proposed controllers under different parameter settings and gait configurations, while remaining secondary to the core findings discussed in the main sections. Again, only some results are shown to not prolong the report.

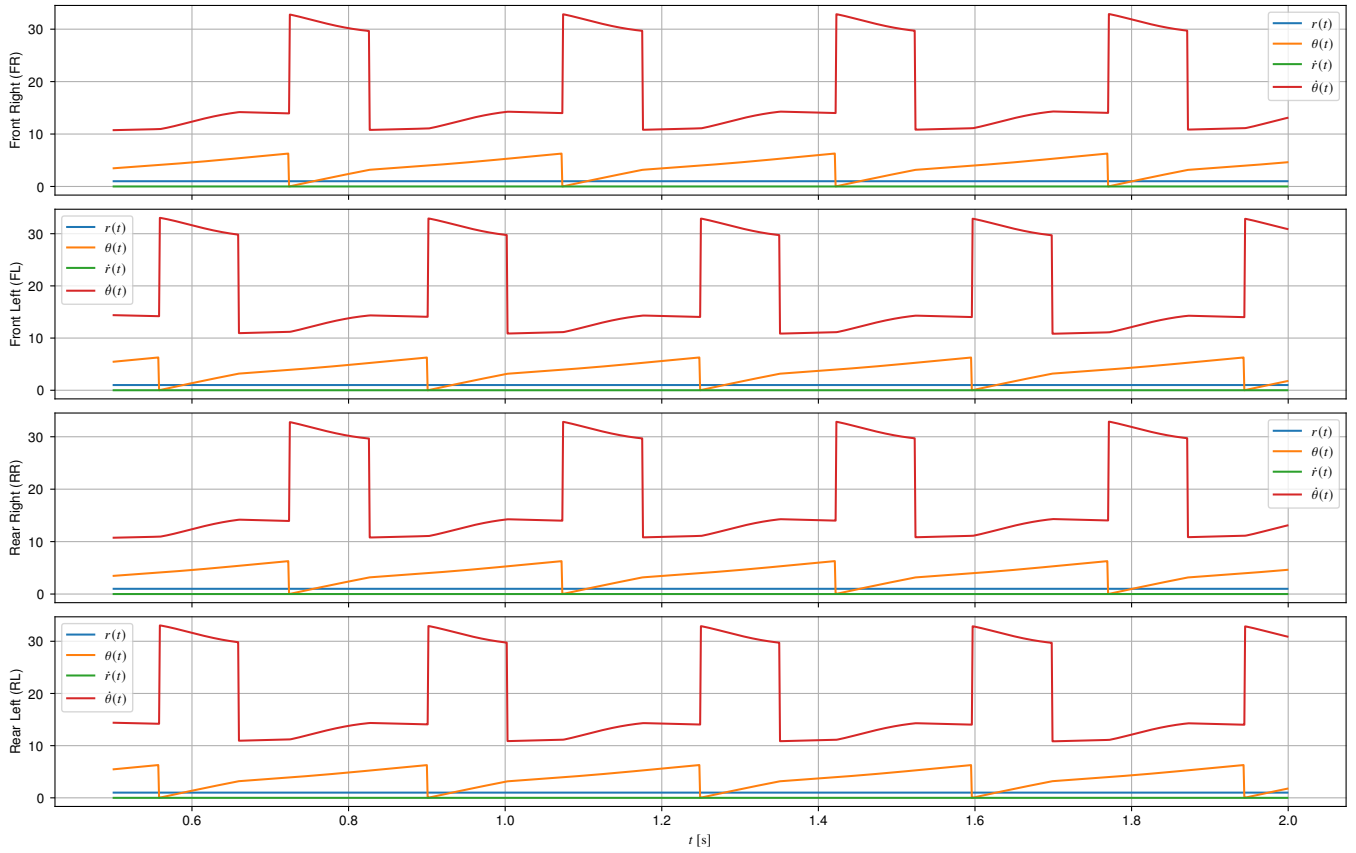


Figure 8: Time evolution of the CPG states. The pace gait is shown.

1) *Fast Trot Gait*: Thus far, we have shown only the slow trot gait, yet the fast trot gait has also been implemented. As the plot of the CPG states would look essentially the same, we choose to only mention the found frequencies and the overall behavior of the robot in this regime. Specifically, the fast trot is simply a result of tripling the stance and swing frequencies of the slow trot to 15 rad/s as for the swing frequency and to 6 rad/s as for the stance frequency. In simulation, the robot behaves borderline unrealistically, moving its legs overall rapidly. Therefore, we think porting this CPG to a real robot might damage the hardware of the A1 quadruped.

2) *Pace Gait*: The pace gait inherits the stance and swing frequencies of the slow trot gait, and therefore differs primarily in its inter-leg coordination rather than in its temporal gait parameters. As a result, any changes in locomotion behavior can be attributed mainly to the altered phase relationships imposed by the coupling matrix. In this context, examining the resulting locomotion metrics becomes particularly informative, as it allows for a direct comparison between gaits that share identical timing characteristics but differ in coordination structure. The corresponding performance metrics for the pace gait are reported in Table 5. In addition, the time evolution of the CPG states shown in Figure 8 clearly reflects the defining characteristics of the pace gait. The ipsilateral leg pairs, namely the front right (FR) and rear right (RR) legs as well as the front left (FL) and rear left (RL) legs, oscillate in phase,

while the two lateral pairs remain in antiphase with respect to each other. This coordination pattern is preserved consistently over successive gait cycles. The observed behavior confirms that the selected coupling structure successfully enforces the intended pace gait and that the resulting dynamics converge to a stable and repeatable limit cycle.

Performance Metric	Symbol	Value	Unit
Minimum body velocity	$\min v_{\text{body}}$	0.16	[m/s]
Maximum body velocity	$\max v_{\text{body}}$	0.52	[m/s]
Duty cycle/ratio	D	0.71	[1]
Stance duration	T_{stance}	0.24	[s]
Swing duration	T_{swing}	0.10	[s]
Stride duration	T_{stride}	0.35	[s]
Cost of transport	CoT	1.16	[1]

Table 5: Performance metrics for the pace gait.

The reported metrics indicate stable and consistent locomotion for the pace gait. The duty ratio of $D = 0.71$ confirms a stance-dominated gait. This behavior is further supported by the relative proportions of stance and swing durations, where the stance phase occupies a larger fraction of the stride. The resulting stride duration of $T_{\text{stride}} = 0.35$ s corresponds to a low-frequency gait consistent with the selected CPG parameters. The measured cost of transport lies within the expected range for CPG-driven locomotion without explicit energetic optimization, indicating reasonable energetic effi-

ciency given the simplicity of the control framework. Overall, the locomotion metric for the pace gait not only extend but also support the slow trot gait shown in the main body.

3) *Bound and Walk Gaits*: Even though the bound and walk gaits were implemented as part of the CPG framework, their resulting locomotion performance was found to be suboptimal compared to the other gaits considered. In addition, given the overall length of the report, we choose not to present detailed results for these gaits. Our own qualitative observations indicate that the expected inter-leg coordination patterns are correctly realized; however, the resulting locomotion remains less stable and efficient than for the trot and pace gaits.

APPENDIX B AI DECLARATION

Hereby, we declare that generative AI was used for this project, yet we tried to keep the use of it limited. The two main aspects in which AI was used include code refactoring and sentence rephrasing, which we shall detail further in the following text.

A. Code Refactoring

We start with refactoring, which is the most obvious and most helpful use of generative AI for a project like this. Even if this time, we have not refactored the entire codebase, refactoring itself is a tedious task for all software developers. Hence, using generative AI for such task really improved our productivity and allowed us to explore ideas more quickly, while keeping the functionality of the code we have written unchanged. It is because of the aforementioned reasons we take the liberty to say that generative AI was helpful in this case, even if we had to come in and refactor a lot of the code by hand.

Sample Prompt

Could you refactor this code block so it is more human readable and remains a clear separation of concerns while keeping the functionality exactly as is?

<Some code>

Sample Answer

Here's a cleaner, more structured version that preserves behavior exactly while the clarifying general flow:

<Some refactored code>

B. Sentence Rephrasing

Getting our ideas across in text has not always been as smooth as we would have liked. At some points during work, we have stumbled upon reusing the same words multiple times in a single sentence and other pitfalls present in academic writing. In order to make our point come across in a more concise way or to make some sentences more understandable to the reader, we have used generative AI on some occasions. Yet, we felt like, more often than not, it made sentences even more complex, and potentially misleading. Therefore, we have used the responses merely as inspiration for our own text.

Sample Prompt

Could you rephrase this sentence?

<Some sentence>

Sample Answer

Sure! Here is a polished alternative:

<Some alternatives>

C. Other Use Cases

When it comes to other use cases of generative AI in this work, we have used it for a very small number of tasks besides the ones we have already mentioned, e.g., fixing minor bugs (good) and suggesting typical variable values (suboptimal).

D. Summary

Overall, we are proud to state that, while AI was used, the majority of the code and the entirety of this report were written by hand. We have distanced ourselves from copy-pasting code or sentence suggestions as much as possible, hence why we think the learning experience has stayed the same as if no generative AI was used at all. This also comes down to the fact that AI was mostly used for repetitive tasks, i.e., code refactoring and sentence rephrasing. Finally, AI has made the project nor easier, nor harder to complete. We think it made it more enjoyable as one could spend less time with repetitive, tedious tasks, and focus on the core concepts of the course.